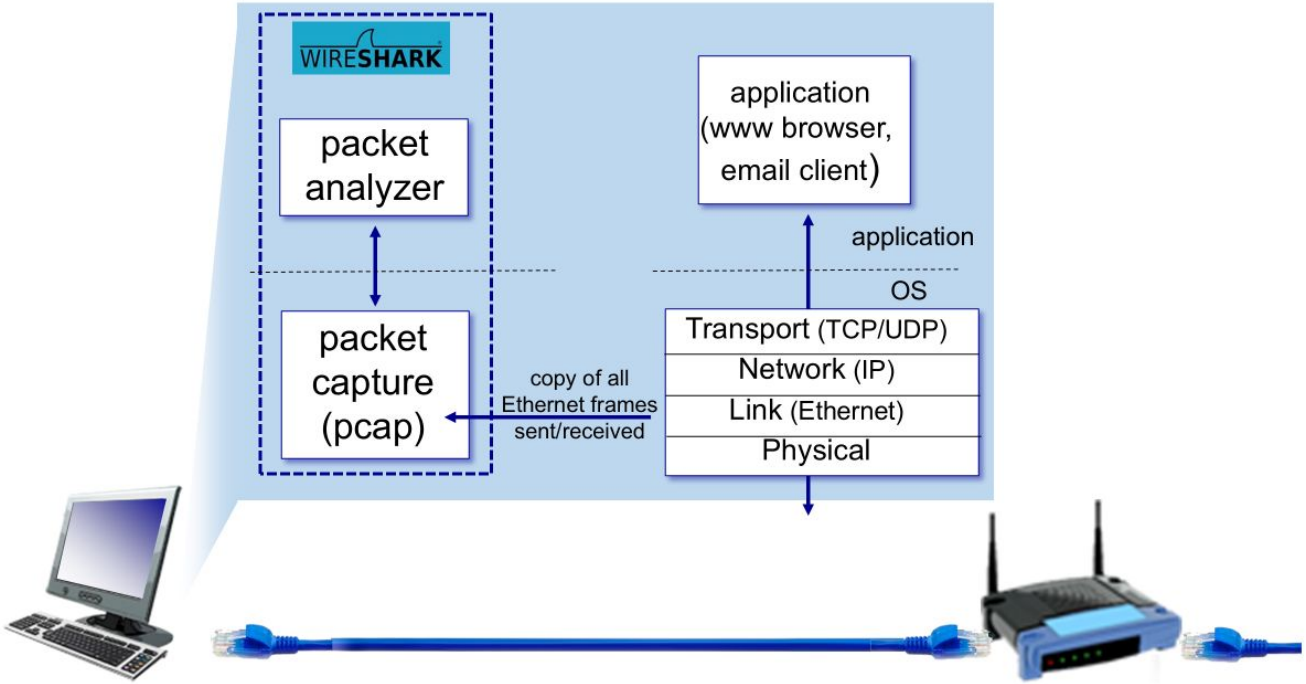
# Lab 4 - Wireshark

# How Wireshark works?




Source: Top-down approach slides: Introduction: 1-9

# Objectives

- To get Hands on experience on Wireshark
- You will be able to capture and analyze packets using wireshark
- You will be able to use wireshark filters and statistics feature for effective analysis
- Able to generate I/O graph and get other network statistics parameters.

# INTRODUCTION

Wireshark is an open-source network packet analyzer that captures and inspects network traffic in real-time.

**Use Case**

- Most common task is network forensics
- Troubleshoot network problems
- Examine network related security issues
- Debug protocol implementation

Generally used by Network Administrators and Cybersecurity Professionals

# INSTALLATION

**Platforms:** Available for Windows, macOS, Linux

**Steps:**

For Ubuntu/Linux: **sudo apt-get install wireshark**

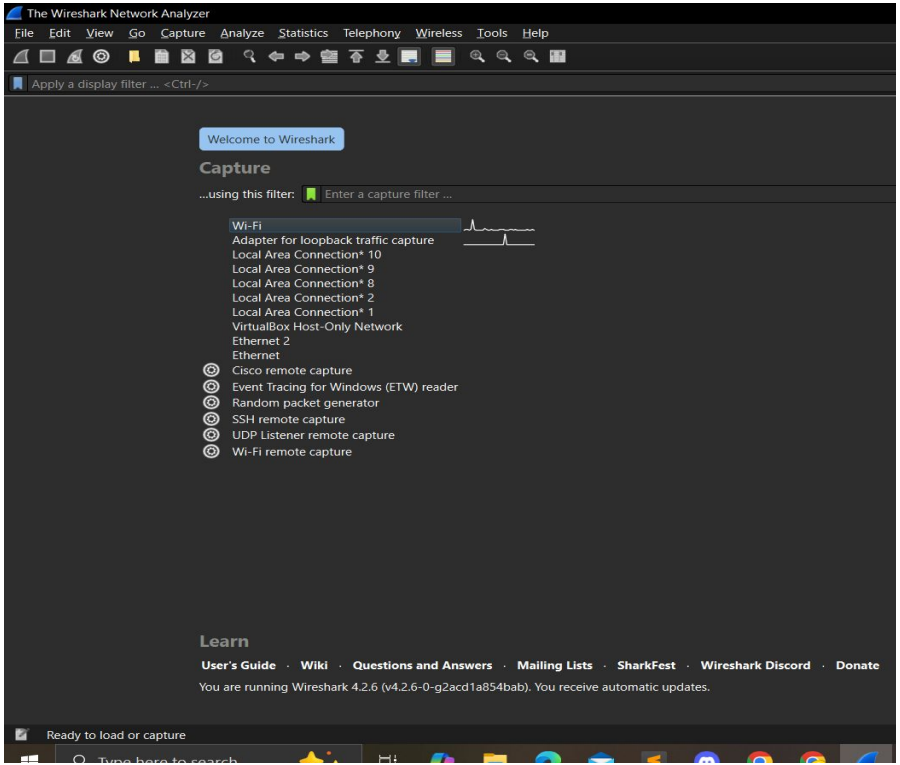
**OR**

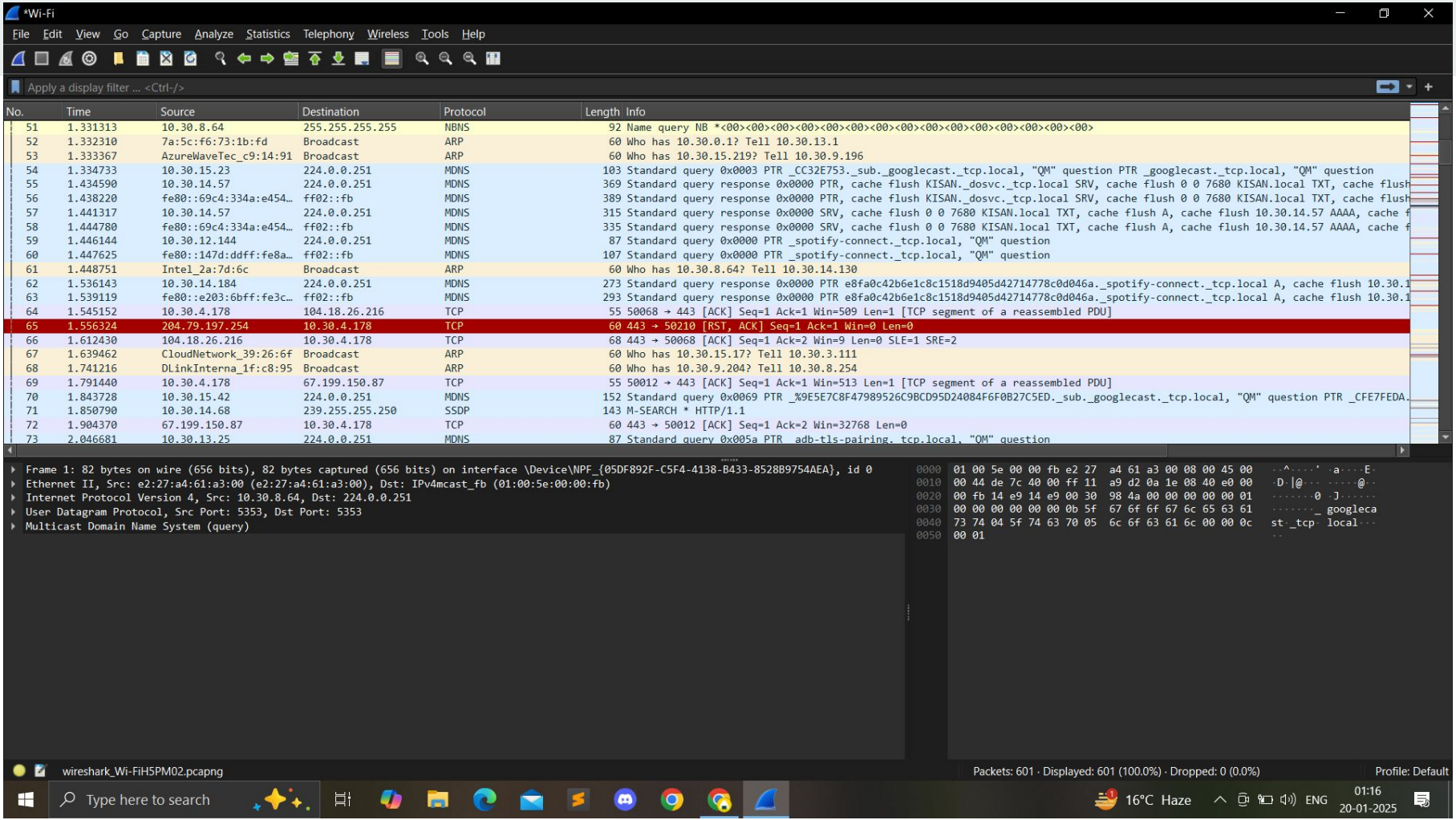
- Download from the official website (www.wireshark.org)
- Run the installer
- Follow installation prompts

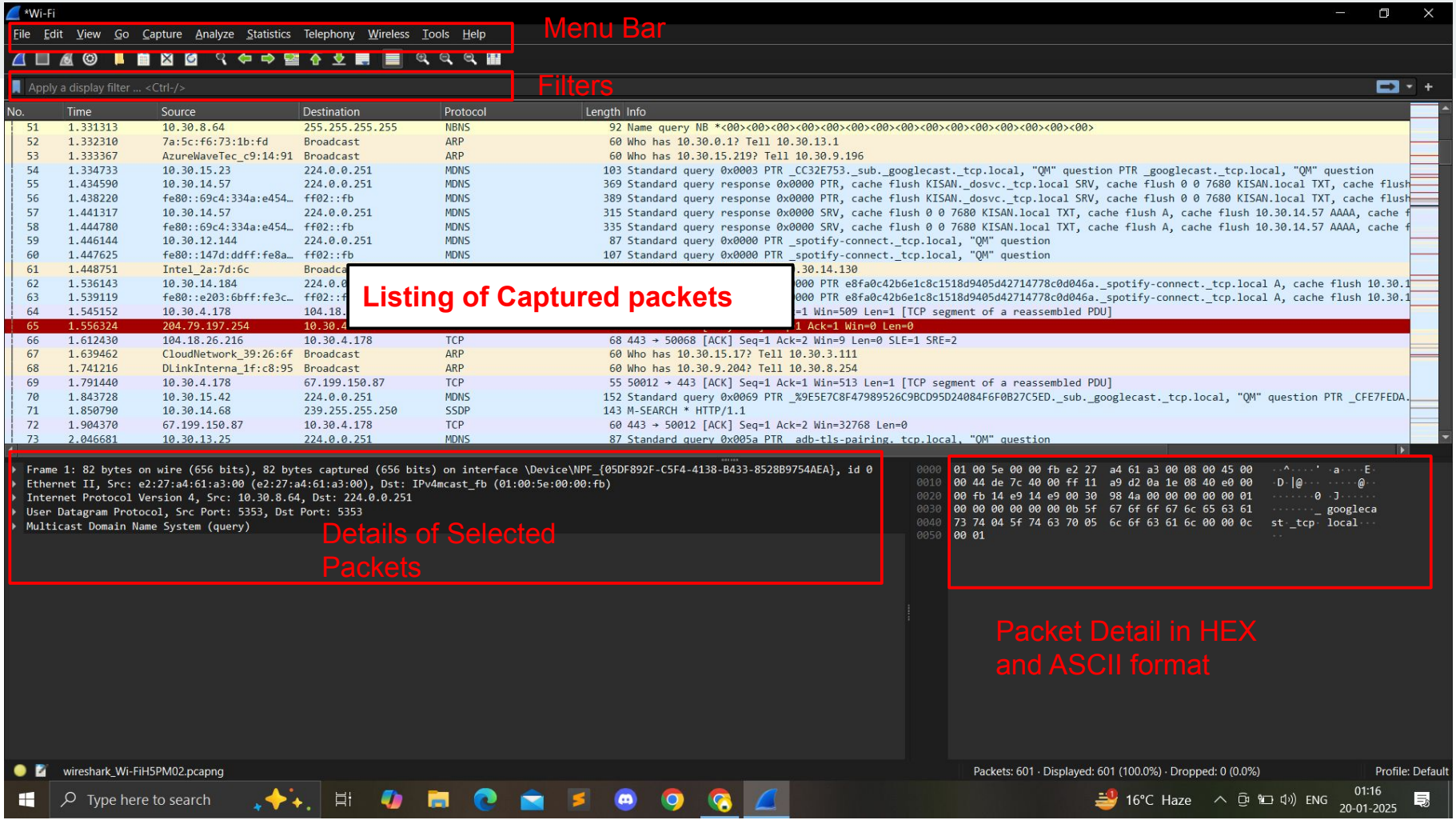
Tip: Install WinPcap or npcap for Windows

File Format: **.pcap** (Packet CAPture)

# On Start Interface




*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
51 1.331313 10.30.8.64 255.255.255.255 NBNS 92 Name query NB *<00><00><00><00><00><00><00><00><00><00><00><00><00><00><00>
52 1.332310 7a:5c:f6:73:1b:fd Broadcast ARP 60 Who has 10.30.0.1? Tell 10.30.13.1
53 1.333367 AzureWaveTec_c9:14:91 Broadcast ARP 60 Who has 10.30.15.219? Tell 10.30.9.196
54 1.334733 10.30.15.23 224.0.0.251 MDNS 103 Standard query 0x0003 PTR _CC32E753._sub._googlecast._tcp.local, "QM" question PTR _googlecast._tcp.local, "QM" question
55 1.434590 10.30.14.57 224.0.0.251 MDNS 369 Standard query response 0x0000 PTR, cache flush KISAN._dosvc._tcp.local SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush
56 1.438220 fe80::69c4:334a:e454… ff02::fb MDNS 389 Standard query response 0x0000 PTR, cache flush KISAN._dosvc._tcp.local SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush
57 1.441317 10.30.14.57 224.0.0.251 MDNS 315 Standard query response 0x0000 SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush A, cache flush 10.30.14.57 AAAA, cache f
58 1.444780 fe80::69c4:334a:e454… ff02::fb MDNS 335 Standard query response 0x0000 SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush A, cache flush 10.30.14.57 AAAA, cache f
59 1.446144 10.30.12.144 224.0.0.251 MDNS 87 Standard query 0x0000 PTR _spotify-connect._tcp.local, "QM" question
60 1.447625 fe80::147d:ddff:fe8a… ff02::fb MDNS 107 Standard query 0x0000 PTR _spotify-connect._tcp.local, "QM" question
61 1.448751 Intel_2a:7d:6c Broadcast ARP 60 Who has 10.30.8.64? Tell 10.30.14.130
62 1.536143 10.30.14.184 224.0.0.251 MDNS 273 Standard query response 0x0000 PTR e8fa0c42b6e1c8c1518d9405d42714778c0d046a._spotify-connect._tcp.local A, cache flush 10.30.1
63 1.539119 fe80::e203:6bff:fe3c… ff02::fb MDNS 293 Standard query response 0x0000 PTR e8fa0c42b6e1c8c1518d9405d42714778c0d046a._spotify-connect._tcp.local A, cache flush 10.30.1
64 1.545152 10.30.4.178 104.18.26.216 TCP 55 50068 → 443 [ACK] Seq=1 Ack=1 Win=509 Len=1 [TCP segment of a reassembled PDU]
65 1.556324 204.79.197.254 10.30.4.178 TCP 60 443 → 50210 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
66 1.612430 104.18.26.216 10.30.4.178 TCP 68 443 → 50068 [ACK] Seq=1 Ack=2 Win=9 Len=0 SLE=1 SRE=2
67 1.639462 CloudNetwork_39:26:6f Broadcast ARP 60 Who has 10.30.15.17? Tell 10.30.3.111
68 1.741216 DLinkInterna_1f:c8:95 Broadcast ARP 60 Who has 10.30.9.204? Tell 10.30.8.254
69 1.791440 10.30.4.178 67.199.150.87 TCP 55 50012 → 443 [ACK] Seq=1 Ack=1 Win=513 Len=1 [TCP segment of a reassembled PDU]
70 1.843728 10.30.15.42 224.0.0.251 MDNS 152 Standard query 0x0069 PTR _%9E5E7C8F47989526C9BCD95D24084F6F0B27C5ED._sub._googlecast._tcp.local, "QM" question PTR _CFE7FEDA.
71 1.850790 10.30.14.68 239.255.255.250 SSDP 143 M-SEARCH * HTTP/1.1
72 1.904370 67.199.150.87 10.30.4.178 TCP 60 443 → 50012 [ACK] Seq=1 Ack=2 Win=32768 Len=0
73 2.046681 10.30.13.25 224.0.0.251 MDNS 87 Standard query 0x005a PTR _adb-tls-pairing._tcp.local, "QM" question
Frame 1: 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF_{05DF892F-C5F4-4138-B433-8528B9754AEA}, id 0
Ethernet II, Src: e2:27:a4:61:a3:00 (e2:27:a4:61:a3:00), Dst: IPv4mcast_fb (01:00:5e:00:00:fb)
Internet Protocol Version 4, Src: 10.30.8.64, Dst: 224.0.0.251
User Datagram Protocol, Src Port: 5353, Dst Port: 5353
Multicast Domain Name System (query)
0000 01 00 5e 00 00 fb e2 27 a4 61 a3 00 08 00 45 00
0010 00 44 de 7c 40 00 ff 11 a9 d2 0a 1e 08 40 e0 00
0020 00 fb 14 e9 14 e9 00 30 98 4a 00 00 00 00 00 01
0030 00 00 00 00 00 00 0b 5f 67 6f 6f 67 6c 65 63 61
0040 73 74 04 5f 74 63 70 05 6c 6f 63 61 6c 00 00 0c
0050 00 01
wireshark_Wi-FiH5PM02.pcapng
Packets: 601 · Displayed: 601 (100.0%) · Dropped: 0 (0.0%)
Profile: Default
Type here to search
16°C Haze
ENG
01:16
20-01-2025


*Wi-Fi
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Menu Bar
Apply a display filter ... <Ctrl-/>
Filters
No. Time Source Destination Protocol Length Info
51 1.331313 10.30.8.64 255.255.255.255 NBNS 92 Name query NB *<00><00><00><00><00><00><00><00><00><00><00><00><00><00><00><00>
52 1.332310 7a:5c:f6:73:1b:fd Broadcast ARP 60 Who has 10.30.0.1? Tell 10.30.13.1
53 1.333367 AzureWaveTec_c9:14:91 Broadcast ARP 60 Who has 10.30.15.219? Tell 10.30.9.196
54 1.334733 10.30.15.23 224.0.0.251 MDNS 103 Standard query 0x0003 PTR _CC32E753._sub._googlecast._tcp.local, "QM" question PTR _googlecast._tcp.local, "QM" question
55 1.434590 10.30.14.57 224.0.0.251 MDNS 369 Standard query response 0x0000 PTR, cache flush KISAN._dosvc._tcp.local SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush
56 1.438220 fe80::69c4:334a:e454... ff02::fb MDNS 389 Standard query response 0x0000 PTR, cache flush KISAN._dosvc._tcp.local SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush
57 1.441317 10.30.14.57 224.0.0.251 MDNS 315 Standard query response 0x0000 SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush A, cache flush 10.30.14.57 AAAA, cache f
58 1.444780 fe80::69c4:334a:e454... ff02::fb MDNS 335 Standard query response 0x0000 SRV, cache flush 0 0 7680 KISAN.local TXT, cache flush A, cache flush 10.30.14.57 AAAA, cache f
59 1.446144 10.30.12.144 224.0.0.251 MDNS 87 Standard query 0x0000 PTR _spotify-connect._tcp.local, "QM" question
60 1.447625 fe80::147d:ddff:fe8a... ff02::fb MDNS 107 Standard query 0x0000 PTR _spotify-connect._tcp.local, "QM" question
61 1.448751 Intel_2a:7d:6c Broadca .30.14.130
62 1.536143 10.30.14.184 224.0.0 000 PTR e8fa0c42b6e1c8c1518d9405d42714778c0d046a._spotify-connect._tcp.local A, cache flush 10.30.1
63 1.539119 fe80::e203:6bff:fe3c... ff02::f 000 PTR e8fa0c42b6e1c8c1518d9405d42714778c0d046a._spotify-connect._tcp.local A, cache flush 10.30.1
64 1.545152 10.30.4.178 104.18. =1 Win=509 Len=1 [TCP segment of a reassembled PDU]
65 1.556324 204.79.197.254 10.30.4 1 Ack=1 Win=0 Len=0
66 1.612430 104.18.26.216 10.30.4.178 TCP 68 443 → 50068 [ACK] Seq=1 Ack=2 Win=9 Len=0 SLE=1 SRE=2
67 1.639462 CloudNetwork_39:26:6f Broadcast ARP 60 Who has 10.30.15.17? Tell 10.30.3.111
68 1.741216 DLinkInterna_1f:c8:95 Broadcast ARP 60 Who has 10.30.9.204? Tell 10.30.8.254
69 1.791440 10.30.4.178 67.199.150.87 TCP 55 50012 → 443 [ACK] Seq=1 Ack=1 Win=513 Len=1 [TCP segment of a reassembled PDU]
70 1.843728 10.30.15.42 224.0.0.251 MDNS 152 Standard query 0x0069 PTR _%9E5E7C8F47989526C9BCD95D24084F6F0B27C5ED._sub._googlecast._tcp.local, "QM" question PTR _CFE7FEDA.
71 1.850790 10.30.14.68 239.255.255.250 SSDP 143 M-SEARCH * HTTP/1.1
72 1.904370 67.199.150.87 10.30.4.178 TCP 60 443 → 50012 [ACK] Seq=1 Ack=2 Win=32768 Len=0
73 2.046681 10.30.13.25 224.0.0.251 MDNS 87 Standard query 0x005a PTR _adb-tls-pairing._tcp.local, "QM" question
Listing of Captured packets
Frame 1: 82 bytes on wire (656 bits), 82 bytes captured (656 bits) on interface \Device\NPF_{05DF892F-C5F4-4138-B433-8528B9754AEA}, id 0
Ethernet II, Src: e2:27:a4:61:a3:00 (e2:27:a4:61:a3:00), Dst: IPv4mcast_fb (01:00:5e:00:00:fb)
Internet Protocol Version 4, Src: 10.30.8.64, Dst: 224.0.0.251
User Datagram Protocol, Src Port: 5353, Dst Port: 5353
Multicast Domain Name System (query)
Details of Selected Packets
0000 01 00 5e 00 00 fb e2 27 a4 61 a3 00 08 00 45 00 ..^....' .a....E.
0010 00 44 de 7c 40 00 ff 11 a9 d2 0a 1e 08 40 e0 00 .D.|@.... .....@..
0020 00 fb 14 e9 14 e9 00 30 98 4a 00 00 00 00 00 01 .......0 .J......
0030 00 00 00 00 00 00 0b 5f 67 6f 6f 67 6c 65 63 61 ........ _ googleca
0040 73 74 04 5f 74 63 70 05 6c 6f 63 61 6c 00 00 0c st._tcp. local...
0050 00 01 ..
Packet Detail in HEX and ASCII format
wireshark_Wi-FiH5PM02.pcapng
Packets: 601 · Displayed: 601 (100.0%) · Dropped: 0 (0.0%)
Profile: Default
Type here to search
16°C Haze
ENG
01:16
20-01-2025

# TRACE ANALYSIS

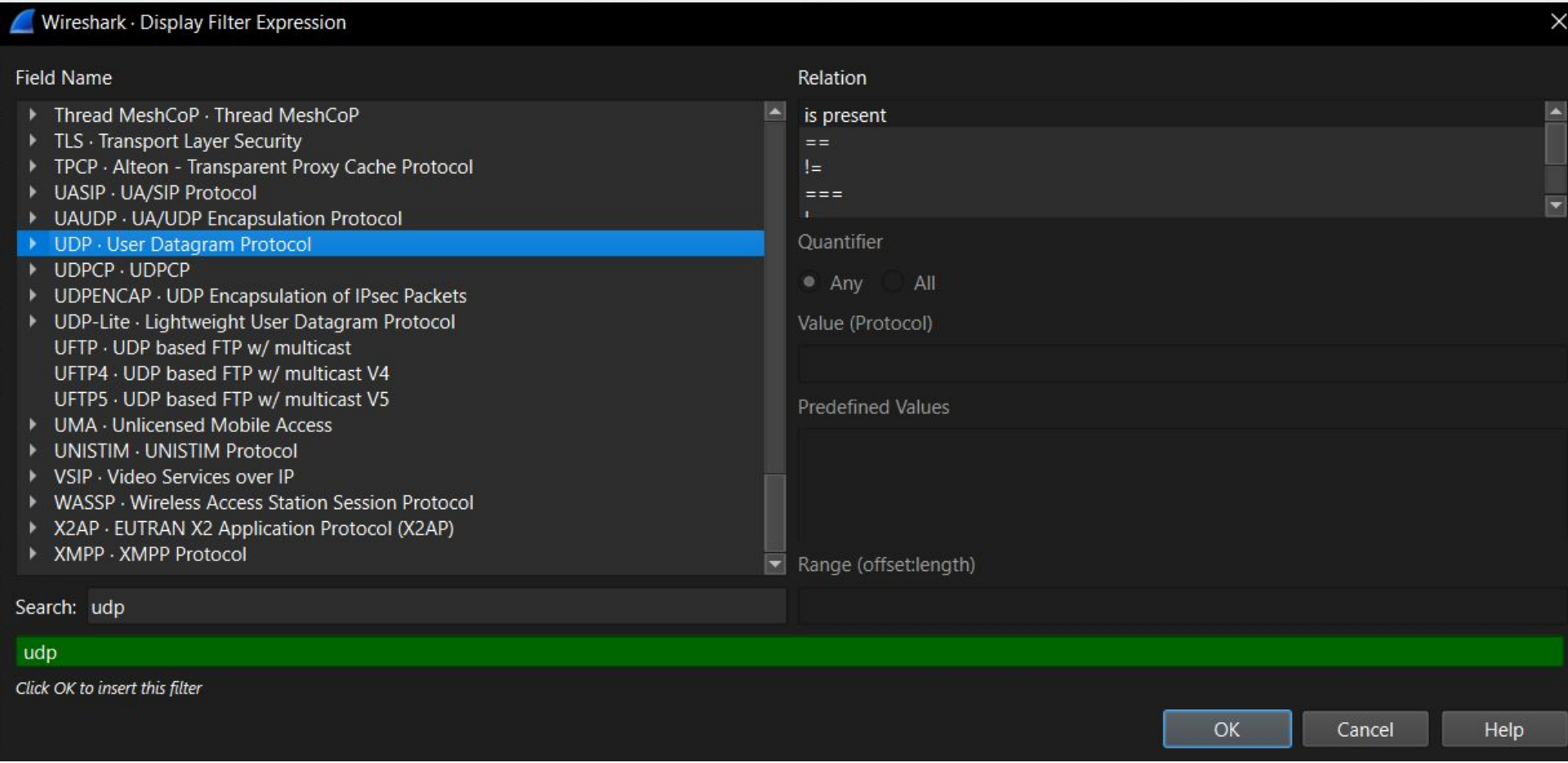
- Displays all of the packets in the trace in the order they were recorded.
- Columns
  - Time – the timestamp at which the packet crossed the interface.
  - Source – the originating host of the packet.
  - Destination – the host to which the packet was sent.
  - Protocol – the highest level protocol that Wireshark can detect.
  - Length – the length in bytes of the packet on the wire.
  - Info – an informational message pertaining to the protocol in the protocol column

# FILTERS

**Purpose**: To filter/isolate the relevant packets out of all captured packets for analysis.

- Filters are evaluated against each packet
- Can either be manually constructed, composed via expression menu or composed based on selected packet's property.
- Filters can be composed of multiple tests joined with boolean connectives.
  - && - logical conjunction (i.e. AND)
  - || - logical disjunction (i.e OR)
  - ! - logical negation (i.e. NOT)

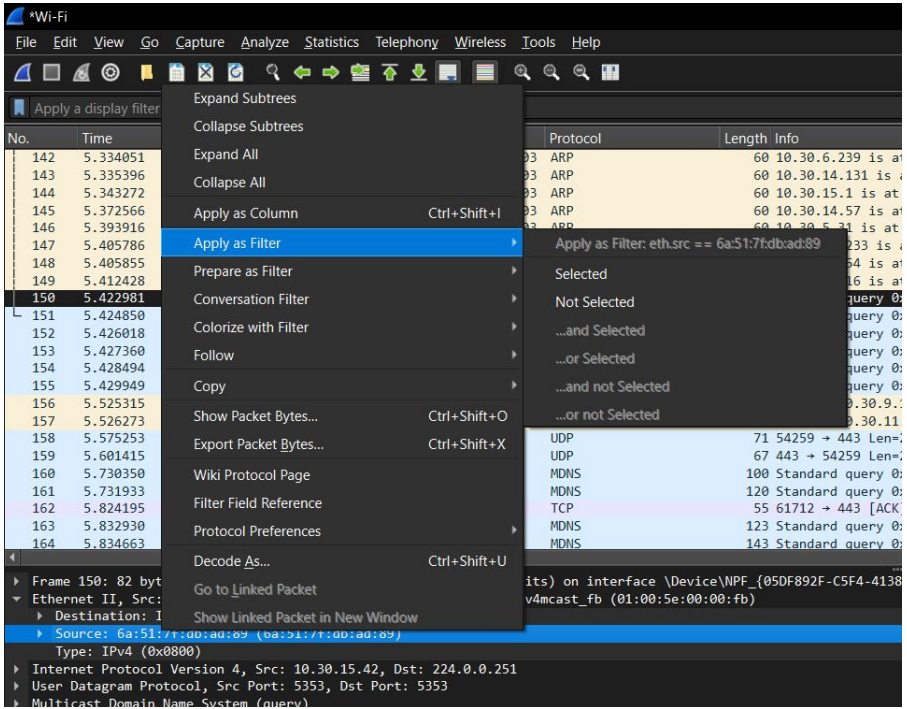
**Expression Menu:** Right click on display filter input field -> Select Display Filter Expression


Wireshark · Display Filter Expression
Field Name
Thread MeshCoP · Thread MeshCoP
TLS · Transport Layer Security
TPCP · Alteon - Transparent Proxy Cache Protocol
UASIP · UA/SIP Protocol
UAUDP · UA/UDP Encapsulation Protocol
UDP · User Datagram Protocol
UDPCP · UDPCP
UDPENCAP · UDP Encapsulation of IPsec Packets
UDP-Lite · Lightweight User Datagram Protocol
UFTP · UDP based FTP w/ multicast
UFTP4 · UDP based FTP w/ multicast V4
UFTP5 · UDP based FTP w/ multicast V5
UMA · Unlicensed Mobile Access
UNISTIM · UNISTIM Protocol
VSIP · Video Services over IP
WASSP · Wireless Access Station Session Protocol
X2AP · EUTRAN X2 Application Protocol (X2AP)
XMPP · XMPP Protocol
Relation
is present
==
!=
===
Quantifier
Any
All
Value (Protocol)
Predefined Values
Range (offset:length)
Search:
udp
udp
Click OK to insert this filter
OK
Cancel
Help

# FILTERS - Selected Packet Property

Filters can be constructed on the basis of individual packets by right clicking on a packet and selecting either:

- Prepare as filter – creates a filter
- Apply as filter – creates a filter and applies it to the trace
- Follow TCP Stream – creates a filter from a TCP packet's stream number and applies it to the trace



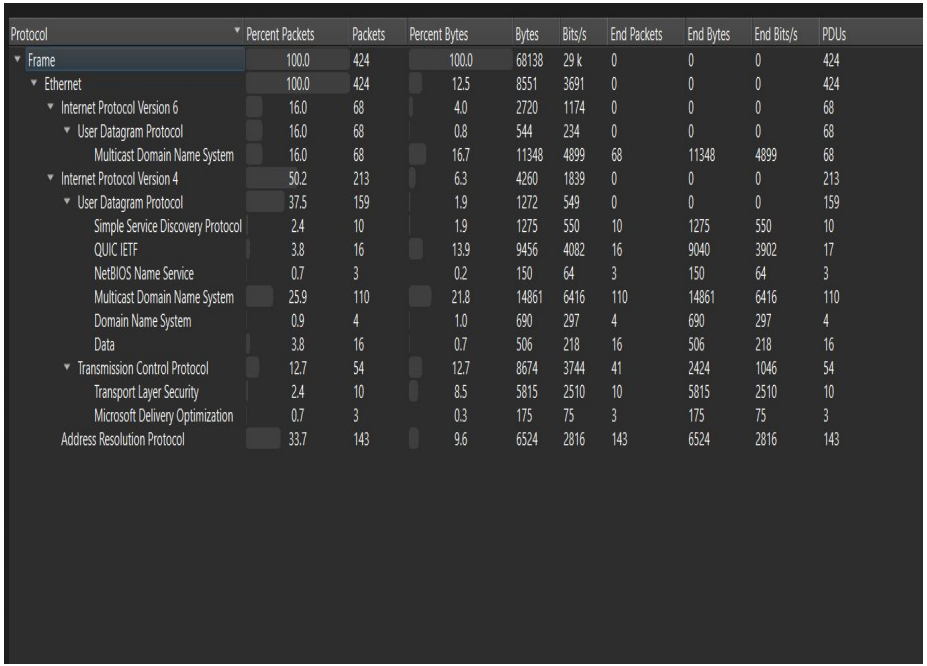
# COMMON FILTERS EXAMPLES

- Filter by IP: "ip.addr == 127.0.0.1 " - Display all IP packets whose source or destination is localhost.
- Filter by Protocol: "http", "dns", "icmp"
- Filter by Port: "tcp.port == 80" or "udp.port == 53"
- Filter by Network Range: "ip.src == 192.168.0.0/24"
- http.request – Display all HTTP requests
- http.request || http.response – Display all HTTP request and responses.
- tcp.len < 100 – Display all TCP packets whose data length is less than 100 bytes.

# STATISTICS

**Statistics -> Protocol Hierarchy**

- Presents descriptive statistics per protocol.
- Useful for determining the types, amounts, and relative proportions of protocols within a trace.

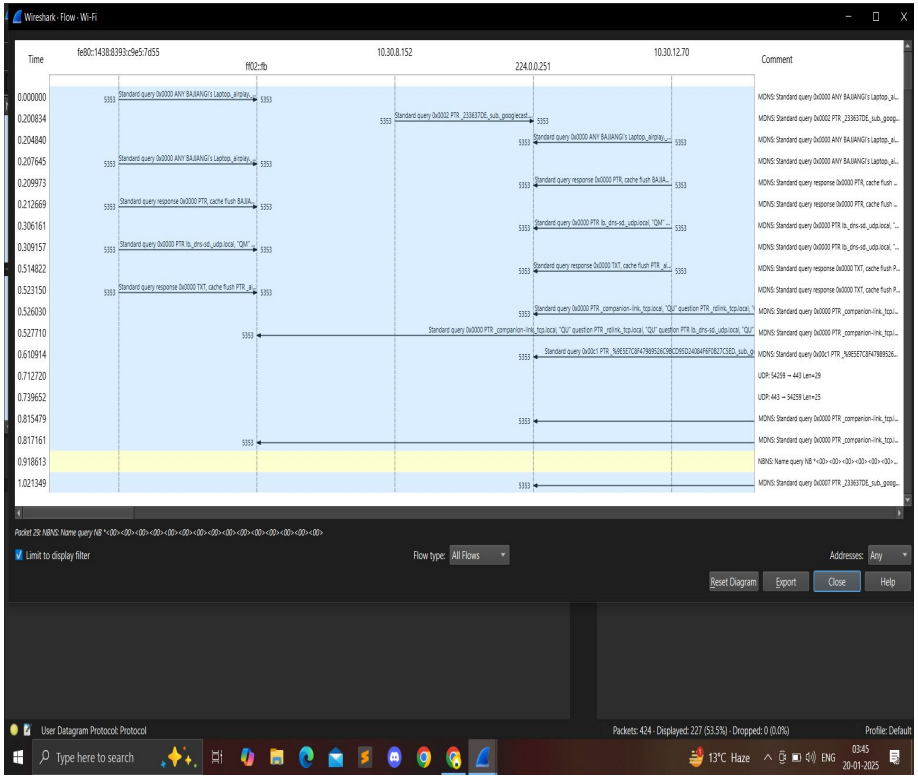


| Protocol | Percent Packets | Packets | Percent Bytes | Bytes | Bits/s | End Packets | End Bytes | End Bits/s | PDUs |
|---|---|---|---|---|---|---|---|---|---|
| Frame | 100.0 | 424 | 100.0 | 68138 | 29 k | 0 | 0 | 0 | 424 |
| Ethernet | 100.0 | 424 | 12.5 | 8551 | 3691 | 0 | 0 | 0 | 424 |
| Internet Protocol Version 6 | 16.0 | 68 | 4.0 | 2720 | 1174 | 0 | 0 | 0 | 68 |
| User Datagram Protocol | 16.0 | 68 | 0.8 | 544 | 234 | 0 | 0 | 0 | 68 |
| Multicast Domain Name System | 16.0 | 68 | 16.7 | 11348 | 4899 | 68 | 11348 | 4899 | 68 |
| Internet Protocol Version 4 | 50.2 | 213 | 6.3 | 4260 | 1839 | 0 | 0 | 0 | 213 |
| User Datagram Protocol | 37.5 | 159 | 1.9 | 1272 | 549 | 0 | 0 | 0 | 159 |
| Simple Service Discovery Protocol | 2.4 | 10 | 1.9 | 1275 | 550 | 10 | 1275 | 550 | 10 |
| QUIC IETF | 3.8 | 16 | 13.9 | 9456 | 4082 | 16 | 9040 | 3902 | 17 |
| NetBIOS Name Service | 0.7 | 3 | 0.2 | 150 | 64 | 3 | 150 | 64 | 3 |
| Multicast Domain Name System | 25.9 | 110 | 21.8 | 14861 | 6416 | 110 | 14861 | 6416 | 110 |
| Domain Name System | 0.9 | 4 | 1.0 | 690 | 297 | 4 | 690 | 297 | 4 |
| Data | 3.8 | 16 | 0.7 | 506 | 218 | 16 | 506 | 218 | 16 |
| Transmission Control Protocol | 12.7 | 54 | 12.7 | 8674 | 3744 | 41 | 2424 | 1046 | 54 |
| Transport Layer Security | 2.4 | 10 | 8.5 | 5815 | 2510 | 10 | 5815 | 2510 | 10 |
| Microsoft Delivery Optimization | 0.7 | 3 | 0.3 | 175 | 75 | 3 | 175 | 75 | 3 |
| Address Resolution Protocol | 33.7 | 143 | 9.6 | 6524 | 2816 | 143 | 6524 | 2816 | 143 |

# STATISTICS

## Statistics -> Flow Graph

- Generates a sequence graph for the selected traffic.
- Useful for understanding seq. and ack.

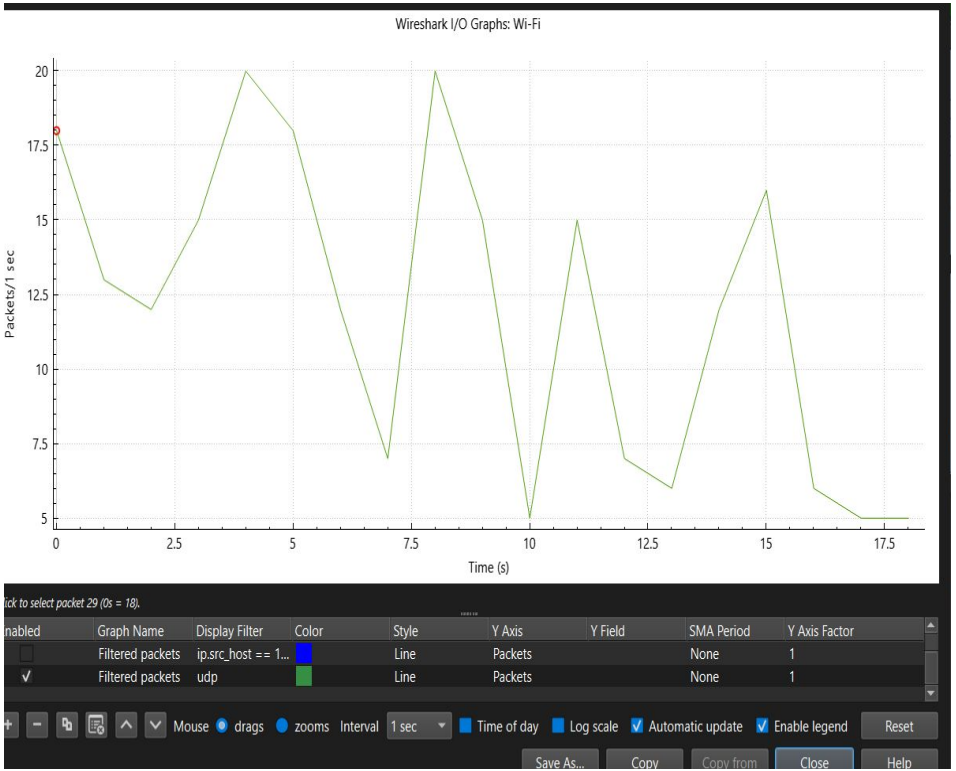


# STATISTICS

**Statistics -> io graph**

- Used to generate I/O Graph
- Lets you plot packet and protocol data in a variety of ways.

https://www.wireshark.org/docs/wsug_html_chunked/ChStatIOGraphs.html



# TO DOs

- Install and setup Wireshark
- Open wireshark and capture traffic for 5-10 minutes
    - During traffic capture, visit iitbhilai.ac.in, send and email to any mail id using mailbox, download “Student Handbook” from iitbhilai.ac.in and watch a video in youtube.com
- Identify all the activities performed by you using the packet captured in wireshark
- Use filters to isolate all the packets related to a particular activity
- Explore the statistics menu of wireshark as well and identify average throughput, first and last packet for each of the activity performed.
- Generate IO graph for each activity
- Create flow graph for dns traffic and explain the graph
- Identify total packets and average packet length per protocol

# Reference/User Guide

https://www.wireshark.org/docs/wsug_html_chunked/



# THANK YOU